

Use Cases

1. **Scenario:** Platform and Market Initialization

- **Actors:** System Admin / System.
- **Trigger:** System Admin (or automated deployment script) starts the platform to open the market for users.
- **Preconditions:**
 - i. The platform and market are currently closed or offline.
 - ii. Configuration for external services is available.
- **Parameters:**
 - i. payment_service_credentials
 - ii. ticketing_service_credentials
- **Main Flow (Success):**
 - i. **System:** Begins the boot sequence to initialize the ticketing platform and the market.
 - ii. **System:** Attempts to establish an active connection with the external **payment** service.
 - iii. **System:** Attempts to establish an active connection with the external **ticket issuing/supply** service.
 - iv. **System:** Validates that at least one System Admin is defined in the database.
 - v. **System:** Ensures all integrity rules and correctness constraints are met.
 - vi. **System:** Transitions the platform and market state to "Active/Open".
- **Alternative Flows (Exceptions):**
 - i. External payment service or ticketing service is unreachable.
>> The system will halt the initialization to prevent integrity violations, log a critical error, and the market will remain closed.
 - ii. No System Admin is defined in the system.
>> The system will pause initialization and prompt for the creation of a default System Admin, as the market cannot open without one.

2. **Scenario:** System Notifications Dispatch (Real-time & Delayed)

- **Actors:** System (Primary), Member (Secondary).
- **Trigger:** A system event occurs that requires user attention (e.g., successful purchase, event sold out, refund issued, role changed).
- **Preconditions:**
 - i. The platform is active and running.
 - ii. The triggering business logic (e.g., ticket inventory reached zero) has successfully completed.
- **Parameters:**
 - i. target_member_id
 - ii. notification_type
 - iii. message_content
- **Main Flow (Success):**
 - i. **System:** Detects a business event that requires notifying specific users.
 - ii. **System:** Identifies the relevant target Member(s) (e.g., the buyer of a ticket, or the producer of an event).
 - iii. **System:** Checks the current connection status (online/offline) of the target Member.
 - iv. **System:** Detects that the target Member is currently online and active on the platform.
 - v. **System:** Transmits the real-time notification to the Member's active session.
- **Alternative Flows (Exceptions/Variations - Delayed):**
 - i. The target Member is offline (Not connected).
 - >> The system will save the notification in the database as a "delayed notification" linked to the Member's account. Upon the Member's next successful login authentication, the system will automatically retrieve and transmit the saved notification to the new session.

3. **Scenario:** Virtual Queue and Load Management

- **Actors:** System (Primary), Visitor (Guest / Member - Secondary).
- **Trigger:** A visitor requests to enter the ticket selection/reservation phase for a specific event, but the concurrent user load for that event has exceeded the defined system threshold.
- **Preconditions:**
 - i. The platform and market are active.
 - ii. The specific event is active and has available tickets.
 - iii. The number of active users currently selecting/reserving tickets for this event is at maximum capacity (threshold crossed).
- **Parameters:**
 - i. event_id
 - ii. visitor_id
- **Main Flow (Success):**
 - i. **Visitor:** Requests to initiate the ticket selection process for event_id.
 - ii. **System:** Evaluates the current load against the defined threshold.
 - iii. **System:** Detects the threshold is exceeded, initializes a Virtual Queue if one is not already active, and assigns the Visitor to the queue.
 - iv. **System:** Provides the Visitor with their current queue status (e.g., estimated wait time or current position in line).
 - v. **System:** Continuously monitors the active load. As capacity frees up, it releases a batch of waiting Visitors from the queue.
 - vi. **System:** Identifies that it is the current Visitor's turn and grants them a temporary, time-limited authorization to enter the selection process.
 - vii. **System:** Notifies the Visitor in real-time that their turn has arrived and grants access to the event's ticket inventory to begin selection.
- **Alternative Flows (Exceptions):**
 - i. Visitor inactivity after authorization.
 - >> The system will monitor the time since authorization was

granted. If the Visitor does not perform an action within the defined time limit, the system will expire their authorization and remove them from the queue/selection phase to free up space.

- ii. The event becomes "Sold Out" while visitors are in the queue.
>> The system will notify all remaining queued Visitors that the inventory has been depleted (Sold Out), and automatically dissolve the queue.

4. Scenario: Guest Registration

- **Actors:** Guest (Primary), System(Secondary).
- **Trigger:** A guest visitor requests to register for a new account on the platform to become a Member.
- **Preconditions:**
 - i. The platform is active.
 - ii. The visitor is currently recognized as a Guest (not logged in).
- **Parameters:**
 - i. user_details (e.g., full name, unique email/username, password)
- **Main Flow (Success):**
 - i. **System:** Receives the registration request
 - ii. **System:** validates the user_details (e.g., ensures the username/email is unique in the system).
 - iii. **System:** Creates a new Member record in the database.
 - iv. **System:** Returns a success message to the visitor, indicating they are now registered.
- **Alternative Flows (Exceptions):**
 - i. The provided unique identifiers (e.g., username/email) already exist in the system.
>> The system rejects the registration, displays an error message, and prompts the visitor to choose different identifiers or log in.

5. Scenario: Member Login

- **Actors:** Guest, System.
- **Trigger:** A guest attempts to authenticate and log into the platform.
- **Preconditions:**
 - i. The platform is active.

- ii. The visitor is currently a Guest (not logged in).
- **Parameters:**
 - i. login_credentials (username/email and password)
- **Main Flow (Success):**
 - i. **System:** Validates the login_credentials .
 - ii. **System:** Authenticates the user and changes their session state from Guest to Logged-in Member.
 - iii. **System:** Grants the visitor the permissions associated with their Member account (and any specific roles they hold).
- **Alternative Flows (Exceptions):**
 - i. The provided credentials do not match any active record.
 >> The system rejects the login attempt and displays an "invalid credentials" error.

6. Scenario: Search and View Event Information

- **Actors:** Visitor (Guest / Member), System.
- **Trigger:** The visitor searches for events or selects a specific event to view its details and venue map.
- **Preconditions:**
 - i. The platform and market are active.
- **Parameters:**
 - i. search_criteria (artist, date, event type, location, price range)
 - ii. event_id (for viewing a specific event)
- **Main Flow (Success):**
 - i. **System:** Processes the search_criteria and retrieves a filtered list of active events from the catalog.
 - ii. **Visitor:** Selects a specific event event_id from the results.
 - iii. **System:** Retrieves the event details, purchase policy, and discount policy.
 - iv. **System:** Retrieves the visual venue map, showing the logical seating/pricing zones and the real-time availability status of seats and general admission areas.
 - v. **System:** Displays the comprehensive event information and the map to the visitor.

- **Alternative Flows (Exceptions):**

- i. No events match the provided search criteria.
>> The system displays an empty result set and prompts the visitor to modify their search.

7. **Scenario:** Ticket Selection and Reservation

- **Actors:** Visitor (Guest / Member), System.
- **Trigger:** The visitor selects tickets (specific seats or general admission quantity) and requests to initiate the reservation process.
- **Preconditions:**
 - i. The requested event is active and has available inventory.
- **Parameters:**
 - i. event_id
 - ii. selection_data (list of seat_ids OR zone_id + quantity)
 - iii. visitor_id
 - iv. lottery_auth_code(required if event is a lottery)
- **Main Flow (Success):**
 - i. **System:** Validates that the visitor does not have an active order for this event.
 - ii. **System:** Checks if the event is configured as a **Lottery**.
 - iii. **System:** (If Lottery) Validates that the Visitor is a registered winner and the provided lottery_auth_code is valid for this visitor_id.
 - iv. **System:** Evaluates the current load against the defined threshold:
 - (If Under Threshold): Grants the visitor immediate authorization to access the ticket inventory.
 - (If Threshold Exceeded): Activates the virtual queue mechanism (see UC 3). Upon successful completion of the queue stage, the system grants the visitor temporary authorization to access the ticket inventory.

- v. **Visitor:** Selects the desired tickets (if not already selected prior to the queue).
- vi. **System:** Checks the current inventory to verify the requested selection_data are still available.
- vii. **System:** Locks (reserves) the requested tickets, temporarily removing them from the global available inventory.
- viii. **System:** Creates a new Active Order for the visitor.
- ix. **System:** Starts a strict countdown timer (e.g., 10 minutes) for the active order.
- x. **System:** Provides the active order details to the visitor with the remaining time to complete checkout.
- **Alternative Flows (Exceptions):**
 - i. The visitor is not a winner or the lottery_auth_code is incorrect/expired.
>>The system Denies access to the reservation process and notifies the visitor.
 - ii. The visitor already has an active order for the event.
>> The system rejects the request, notifying the visitor that they must complete or cancel their current order first.
 - iii. The visitor fails to pass the queue (e.g., timeout due to inactivity, or event becomes Sold Out while waiting - see UC 3).
>> The system aborts the reservation process.
 - iv. The specific seats are locked by another user before the system could complete step 7.
>>The system notifies the visitor and refreshes the map with current availability.

8. Scenario: Manage Active Order

- **Actors:** Visitor (Guest / Member), System.
- **Trigger:** The visitor views their active order and requests to modify it (e.g., remove items or change quantities).
- **Preconditions:**

- i. The visitor holds an Active Order with reserved tickets.
 - ii. The countdown timer has not expired.
- **Parameters:**
 - i. active_order_id
 - ii. modification_request (remove ticket / update quantity)
- **Main Flow (Success):**
 - i. **System:** Validates the request against the remaining available inventory (if increasing quantity).
 - ii. **System:** Updates the contents of the Active Order.
 - iii. **System:** If tickets were removed, immediately releases the corresponding locks, returning the tickets to the global available inventory.
 - iv. **System:** Displays the updated active order summary to the visitor.
- **Alternative Flows (Exceptions):**
 - i. The timer expires while the visitor is managing the order.
 >> The system automatically cancels the active order, releases all locked tickets back to the inventory, and redirects the visitor with a timeout notification.

9. Scenario: Checkout and Payment Completion

- **Actors:** Visitor (Guest / Member), System, Payment Gateway, Ticket Supply Service.
- **Trigger:** The visitor submits payment details to finalize the purchase of their active order.
- **Preconditions:**
 - i. The visitor has an Active Order containing at least one ticket.
 - ii. The active order timer has not expired.
 - iii. External payment and ticketing services are online.
- **Parameters:**
 - i. payment_details
 - ii. discount_code (optional)

- **Main Flow (Success):**

- i. **System:** Evaluates the order against the specific event/company Purchase Policy (e.g., maximum tickets per buyer limits).
- ii. **System:** Calculates the final price by applying the Discount Policy and validating the discount_code if provided.
- iii. **System:** Dispatches a charge request to the Payment Gateway.
- iv. **Payment Gateway:** Approves the transaction.
- v. **System:** Dispatches a ticket generation request to the Ticket Supply Service.
- vi. **Ticket Supply Service:** Issues the tickets/barcodes.
- vii. **System:** Finalizes the order into a Receipt, permanently deducting the tickets from the inventory.
- viii. **System:** Clears the Active Order state and sends a success notification to the visitor.

- **Alternative Flows (Exceptions):**

- i. The purchase violates the Purchase Policy.
>> The system halts checkout, displays a policy error, and allows the user to modify the order (timer keeps running).
- ii. The Payment Gateway declines the transaction.
>> The system notifies the visitor of payment failure. The order remains active until the timer expires.
- iii. The Payment Gateway times out or does not respond within the acceptable timeframe.
>> The system aborts the payment transaction attempt to prevent hanging states, notifies the visitor of the communication error, and allows them to retry the checkout process provided the Active Order countdown timer has not yet expired.
- iv. Ticket Supply Service fails after payment is processed.
>> The system triggers an automatic Refund via the Payment

Gateway, releases the tickets back to inventory, and notifies the visitor of the cancellation due to a technical error.

10. Scenario: Member Logout with Active Reservation

- **Actors:** Member, System.
- **Trigger:** A logged-in Member initiates a logout or exits the system while they have an active ticket reservation (active order).
- **Preconditions:**
 - i. The user is logged in as a Member.
 - ii. The Member has an active order with a running countdown timer for reserving tickets.
- **Parameters:**
 - i. member_id
 - ii. active_order_id
- **Main Flow (Success):**
 - i. **Member:** Requests to terminate the session.
 - ii. **System:** Demotes the Member's current session to a Guest-Visitor state.
 - iii. **System:** Links the active order and its remaining reservation timer to the member_id in the database.
 - iv. **System:** Maintains the lock on the reserved tickets while the timer is still ticking.
- **Alternative Flows (Exceptions):**
 - i. Members log back in before the timer expires.
 - >> The system detects the active order linked to the member_id upon successful login and redirects the Member to continue the checkout process from the exact point they left off.
 - ii. The reservation timer expires while the Member is logged out.
 - >> The system cancels the order, releases the locked tickets back to the general inventory, and clears the active order state for the member_id.

11. Scenario: Open New Production Company

- **Actors:** Member, System.
- **Trigger:** A Member wants to register a new production company on the platform.
- **Preconditions:**
 - i. The user is logged in as a Member.
- **Parameters:**
 - i. member_id
 - ii. company_details
- **Main Flow (Success):**
 - i. **Member:** Submits a request to register a new production company.
 - ii. **System:** Validates the submitted data (e.g., ensuring the company name doesn't already exist).
 - iii. **System:** Creates the new production company record in the database.
 - iv. **System:** Assigns the "Founder" role to the member_id for this specific company.
 - v. **System:** Grants the Member access to the company's management dashboard.
- **Alternative Flows (Exceptions):**
 - i. Company details are invalid or duplicate.
 - >> The system halts the creation, highlights the errors to the Member, and prompts them to fix the details.

12. Scenario: View Personal Purchase History (Immutable Records)

- **Actors:** Member, System.
- **Trigger:** A Member wants to see their past receipts and current ticket statuses.

- **Preconditions:**
 - i. The user is logged in as a Member.
- **Parameters:**
 - i. member_id
- **Main Flow (Success):**
 - i. **Member:** Requests to view personal purchase history.
 - ii. **System:** Retrieves past purchase records and future event ticket statuses linked to the member_id.
 - iii. **System:** Renders the historical data exactly as it was captured at the time of purchase (ensuring records are completely agnostic to subsequent platform changes like retroactive price drops, event cancellations, or production company closures).
 - iv. **System:** Displays the tickets and receipts to the Member.
- **Alternative Flows (Exceptions):**
 - i. No purchase history exists for the Member.
 - >> The system displays an empty state message indicating that no past orders or tickets were found.

13. Scenario: Update Identifying Details

- **Actors:** Member, System.
- **Trigger:** A Member wants to view or update their personal profile information.
- **Preconditions:**
 - i. The user is logged in as a Member.
- **Parameters:**
 - i. member_id
 - ii. updated_profile_data
- **Main Flow (Success):**
 - i. **Member:** Submits updated identifying details.

- ii. **System:** Validates the integrity of the new data (e.g., valid email format, strong password).
- iii. **System:** Updates the member_id record in the database.
- iv. **System:** Displays a success confirmation to the Member.
- **Alternative Flows (Exceptions):**
 - i. Data validation fails.
 - >> The system rejects the update and informs the Member of the required corrections.

14. Scenario: Register for Purchase Right Lottery

- **Actors:** Member, System.
- **Trigger:** A Member registers in advance for a lottery to purchase tickets for a high-demand event.
- **Preconditions:**
 - i. The user is logged in as a Member.
 - ii. The Production Company has enabled the lottery mechanism for the specific event.
- **Parameters:**
 - i. member_id
 - ii. event_id
- **Main Flow (Success):**
 - i. **Member:** Submits a registration request for the event's lottery.
 - ii. **System:** Validates the Member is not already registered.
 - iii. **System:** Adds the member_id to the lottery pool for the event_id.
 - iv. **System:** Confirms successful registration to the Member.
- **Alternative Flows (Exceptions/Variations - Post-Lottery Draw):**
 - i. The Member wins the lottery draw.
 - >> The system sends a notification to the winning Member containing a unique, time-limited authorization code, allowing them to enter the ticket reservation phase when sales open.

15. Scenario: Create and Configure Event

- **Actors:** Owner/Manager (Primary), System (Secondary)

- **Trigger:** An Owner or authorized Manager wants to add a new event to the production company's catalog, including its venue map configuration.
- **Preconditions:**
 - i. The user is logged in as a Member with an Owner or Manager role for the target production company.
 - ii. The production company is in an "Active" state.
- **Parameters:**
 - i. company_id
 - ii. event_details (name, artist/performer, type, date(s)/show(s), description)
 - iii. venue_map_data (zones, seat layout, pricing per zone, total_capacity)
- **Main Flow (Success):**
 - i. **Owner:** Requests to create and configure a new event.
 - ii. **System:** Validates that all mandatory fields are provided and that the production company is active.
 - iii. **System:** Verifies that the total number of seats/positions defined in the venue_map_data does not exceed the declared total_capacity.
 - iv. **System:** Creates the new event record and links it to the company_id.
 - v. **System:** Generates the visual venue map with the defined zones, seating arrangement, and pricing tiers.
 - vi. **System:** Publishes the event to the platform catalog with an "Active" status, making it visible and searchable to visitors.
- **Alternative Flows (Exceptions):**
 - i. One or more mandatory fields are missing or invalid.
 >> The system rejects the creation, highlights the problematic fields, and prompts the Owner to correct them.

- ii. The production company is currently suspended or closed.
>> The system blocks the creation and displays an error stating the company must be active to create events.

16. **Scenario:** Define Purchase and Discount Policies

- **Actors:** Owner (Primary), System (Secondary).
- **Trigger:** An Owner wants to set or update the commercial rules (purchase restrictions and/or discount schemes) for their production company or a specific event.
- **Preconditions:**
 - i. The user is logged in as a Member with an Owner role for the target production company.
 - ii. The production company is in an “Active” state.
- **Parameters:**
 - i. company_id
 - ii. event_id (optional if omitted, policies apply at the company level)
 - iii. purchase_policy_data (e.g., max tickets per buyer, age restrictions, reserved seat limits)
 - iv. discount_policy_data (discount type: visible / conditional / coupon code, discount value, conditions)
- **Main Flow (Success):**
 - i. **Owner:** Requests to define or update commercial policies, specifying whether the policy applies at the company level or to a specific event.
 - ii. **System:** Validates the policy data for internal consistency (e.g., no conflicting rules between purchase limits and discount conditions).
 - iii. **System:** Saves the policy and links it to the appropriate company_id or event_id scope.

iv. **System:** Confirms the successful policy update to the Owner.

- **Alternative Flows (Exceptions):**

- i. A conflict is detected between purchase and discount policies (e.g., a “buy 5 get 1 free” discount while the purchase policy limits buyers to 4 tickets).

- >> The system rejects the policy, displays a warning describing the conflict, and prompts the owner to resolve it.

- ii. The Owner attempts to modify a policy while active orders exist for the affected event.

- >> The system warns the Owner that changes will only apply to future orders and requires explicit confirmation before saving.

17. **Scenario:** Manage Company Roles (Appoint and Remove Owners and Managers)

- **Actors:** Owner/Founder (Primary), System (Secondary).

- **Trigger:** A Founder or Owner wants to appoint or remove a Manager or Owner for their production company.

- **Preconditions:**

- i. The user is logged in as a Member with an Owner or Founder role for the target production company.

- ii. The target Member to be appointed exists and is a registered platform Member.

- **Parameters:**

- i. company_id

- ii. target_member_id

- iii. role_type (Owner / Manager)

- iv. permissions_list (required only for Manager appointments defines which management actions the Manager is authorized to perform)

- **Main Flow (Success):**

- i. **Owner/Founder:** Submits a role appointment request specifying the target_member_id, role_type, and (for Managers) the permissions_list.
- ii. **System:** Validates the requester's authorization level (e.g., only a Founder may appoint an Owner).
- iii. **System:** Validates that the target Member does not already hold the specified role in this company.
- iv. **System:** Assigns the role and permissions to the target Member and updates the company's role registry.
- v. **System:** Sends a notification to the target Member informing them of their new role.
- **Alternative Flows (Exceptions):**
 - i. A non-Founder Owner attempts to appoint another Owner.
>> The system denies the request and displays an "insufficient permissions" error, as only the Founder may appoint Owners.
 - ii. The target Member already holds the specified role in this company.
>> The system rejects the request and informs the requester that the appointment is redundant.
 - iii. A Founder removes an Owner from the company.
>> The system revokes the Owner's role and all associated permissions, and notifies the removed Owner.
 - iv. An Owner voluntarily resigns from their role.
>> The system confirms the resignation, revokes their permissions, and notifies the Founder. If the resigning Owner is the last Owner (excluding the Founder), the system warns the Founder.

18. **Scenario:** Generate Sales Report and View Purchase History

- **Actors:** Owner/Manager (Primary), System (Secondary).
- **Trigger:** An Owner or authorized Manager wants to review transaction history and revenue data for their production company's events.

- **Preconditions:**
 - i. The user is logged in as a Member with an Owner or authorized Manager role for the target production company.
- **Parameters:**
 - i. company_id
 - ii. filter_type (event / date range)
 - iii. filter_value
- **Main Flow (Success):**
 - i. **Owner:** Requests a sales report based on specified criteria.
 - ii. **System:** Queries the database for all transactions associated with the company_id, filtered by the specified criteria.
 - iii. **System:** Aggregates the data into a report displaying total revenue, ticket counts, and per-event breakdowns.
 - iv. **System:** Displays the report to the Owner.
- **Alternative Flows (Exceptions):**
 - i. No sales records match the provided filter criteria.
 >> The system displays an empty state message indicating 0 results for the applied filters.

19. **Scenario:** Suspend and Reopen Production Company

- **Actors:** Founder (Primary), System (Secondary).
- **Trigger:** The Founder of a production company voluntarily suspends or reopens their own company. (This is distinct from an Admin-forced closure in Part 6.)
- **Preconditions:**
 - i. The user is logged in as the Founder of the target production company.
 - ii. For reopening: the company was previously suspended by the Founder (not closed by a System Admin).
- **Parameters:**
 - i. company_id
 - ii. action_type (suspend / reopen)

- **Main Flow (Success):**

- Founder:** Initiates the suspension command for company_id.
- System:** Changes the production company's operational status to "Suspended."
- System:** Prevents any new event creation or ticket sales under this company. Existing active orders with running countdown timers are allowed to complete their checkout naturally; the suspension takes effect only for new actions.
- System:** Dispatches suspension notifications to all Owners and Managers of the company.
- System:** Logs the action.

- **Main Flow (Success Reopen):**

- Founder:** Initiates the reopen command for a suspended company_id.
- System:** Validates that the company was suspended by the Founder (not closed by a System Admin).
- System:** Changes the company's status back to "Active."
- System:** Dispatches reactivation notifications to all Owners and Managers.

- **Alternative Flows (Exceptions):**

- The Founder attempts to suspend a company that has future events with sold tickets.
>> The system displays a warning listing the affected events and ticket counts, and requires explicit confirmation that mass cancellation and refunds will be initiated before proceeding.
- The Founder attempts to reopen a company that was closed by a System Admin.

>> The system denies the request, displaying an error stating that Admin-closed companies cannot be reopened by the Founder.

20. **Scenario:** Perform Authorized Management Action

- **Actors:** Manager (Primary), System (Secondary).
- **Trigger:** A Manager attempts to perform a management action on behalf of the production company (e.g., editing an event, updating the venue map, modifying policies).
- **Preconditions:**
 - i. The user is logged in as a Member with a Manager role for the target production company.
 - ii. The production company is in an “Active” state.
- **Parameters:**
 - i. company_id
 - ii. action_type (e.g., edit event, update venue map, modify policy)
 - iii. action_data
- **Main Flow (Success):**
 - i. **Manager:** Submits a management action request for company_id.
 - ii. **System:** Verifies that the Manager’s assigned permissions_list includes authorization for the requested action_type.
 - iii. **System:** Executes the action following the same business logic as the corresponding Owner action (as defined in Part 4).
 - iv. **System:** Confirms the successful action to the Manager.
- **Alternative Flows (Exceptions):**
 - i. The Manager does not have the required permission for the requested action.

>> The system denies the request and displays an “insufficient permissions” error, specifying which permission is missing.

- ii. The Owner modifies the Manager's permissions while the Manager is in the middle of an action.
>> The system re-validates permissions at the time of submission. If the permission was revoked, the action is denied and the Manager is notified of the change.

21. **Scenario:** Admin Closes Production Company

- **Actors:** System Admin, System.
- **Trigger:** A System Admin decides to shut down a production company due to violations (e.g., fraud, mass cancellations).
- **Preconditions:**
 - i. The user is logged in as a System Admin.
 - ii. The target production company exists.
- **Parameters:**
 - i. company_id
 - ii. closure_reason
- **Main Flow (Success):**
 - i. System Admin: Initiates the closure command for company_id .
 - ii. **System:** Changes the operational status of the production company to "Closed".
 - iii. **System:** Automatically cancels all active roles, permissions, and subscriptions for users associated with managing this company.
 - iv. **System:** Dispatches closure notifications to the registered owners and managers of the company.
 - v. **System:** Logs the administrative action.
- **Alternative Flows (Exceptions):**
 - i. The company has active future events with sold tickets.
>> The system prompts the Admin with a warning and asks if they want to initiate a mass cancellation and refund process for all future events linked to this company before finalizing the closure.

- ii. The company does not exist.
>> The system returns an error stating the company could not be found.

22. **Scenario:** Admin Removes/Bans Member

- **Actors:** System Admin, System.
- **Trigger:** A System Admin cancels a registered user's subscription/account due to infractions.
- **Preconditions:**
 - i. The user is logged in as a System Admin.
 - ii. The target Member account exists.
- **Parameters:**
 - i. target_member_id
 - ii. ban_reason
- **Main Flow (Success):**
 - i. **System Admin:** Executes the ban/removal command for target_member_id.
 - ii. **System:** Disables the target_member_id account, preventing any future logins.
 - iii. **System:** Scans the database for any production company roles or permissions assigned to this Member.
 - iv. **System:** Automatically revokes all identified roles across all production companies for this user.
 - v. **System:** Logs the administrative action.
- **Alternative Flows (Exceptions):**
 - i. The target Member is the sole "Founder" of an active production company.
>> The system alerts the Admin of the dependency and halts the

automatic role revocation until the Admin manually decides whether to assign a new Founder or close the orphaned production company.

23. **Scenario:** View Global Purchase History

- **Actors:** System Admin, System.
- **Trigger:** A System Admin needs to review transactions across the entire platform.
- **Preconditions:**
 - i. The user is logged in as a System Admin.
- **Parameters:**
 - i. filter_type (Buyer / Production Company / Event)
 - ii. filter_value
- **Main Flow (Success):**
 - i. **System Admin:** Requests global purchase history based on specified criteria.
 - ii. **System:** Queries the central database ledger based on the provided cross-section (e.g., all tickets bought by a specific user, or all sales for a specific event).
 - iii. **System:** Aggregates and displays the comprehensive transaction history to the Admin.
- **Alternative Flows (Exceptions):**
 - No records match the requested cross-section.

The system returns an empty state message indicating 0 results for the applied filters.

User Acceptance Tests - UAT

1. System Acceptance Tests

Test ID	UC Ref	Test Scenario	Procedure (Physical)	Acceptance Criteria (Status & Logs)
UAT-01	UC 1	Successful Initialization	Run boot sequence with valid service credentials and a defined Admin in DB.	SUCCESS Logs: Boot_Success, Services_Connected.
UAT-02	UC 1	Init Fails - Service Down	Run boot sequence while payment or ticketing service is unreachable.	ERROR Logs: Init_Halted, Error_Service_Unreachable
UAT-03	UC 1	Init Fails - No Admin	Run boot sequence with an empty Admin table in the database.	FAIL Logs: Init_Paused, Warning_No_Admin
UAT-04	UC 2	Real-time Notification	Trigger a business event for a target Member who is currently online.	SUCCESS Logs: Notification_Dispatched_Realtime
UAT-05	UC 2	Delayed Notification	Trigger an event for an offline Member, then log in with that Member's account.	SUCCESS Logs: Notification_Saved, Notification_Dispatched_Delayed.

UAT-06	UC 3	Enter Virtual Queue	Request to initiate ticket purchase for an event where the current active load is greater than the defined threshold.	SUCCESS Logs: User_Queued, Load_Threshold_Exceeded.
UAT-07	UC 3	Queue Turn Arrives	Wait in Virtual Queue until capacity frees up and the system grants authorization.	SUCCESS Logs: Queue_Turn_Granted, Auth_Issued.
UAT-08	UC 3	Queue Inactivity Timeout	Wait in queue, get turn authorization, and perform no action until the time limit expires.	FAIL Logs: Auth_Expired, User_Removed_From_Queue.
UAT-09	UC 3	Sold Out While Queued	Trigger "Sold Out" state for the event while the Visitor is waiting in the queue.	FAIL Logs: Queue_Dissolved, Event_Sold_Out.
UAT-10	UC 4	Successful Registration	Run registration process with valid and unique details (name, email, password).	SUCCESS Logs: Member_Registered.
UAT-11	UC 4	Registration Duplicate Details	Run the registration process with an email or username that already exists in the database.	ERROR Logs: Registration_Failed_Duplicate.
UAT-12	UC 5	Successful Login	Submit valid email and password of an existing member.	SUCCESS Logs: Login_Success.

UAT-13	UC 5	Invalid Login Credentials	Submit an incorrect password or non-existent username.	ERROR Logs: Login_Failed.
UAT-14	UC 6	Search With Results	Execute a search for an active event or artist present in the catalog.	SUCCESS Logs: Search_Success.
UAT-15	UC 6	Search Empty Results	Execute a search with a string that does not match any event	SUCCESS Logs: Search_Empty.
UAT-16	UC 7	Successful Reservation (Regular Event)	Select available tickets for a non-lottery event under threshold with no existing order	SUCCESS Logs: Tickets_Locked, Order_Created.
UAT-17	UC 7	Successful Lottery Reservation	Submit a valid lottery auth code for an event won by the Member.	SUCCESS Logs: Lottery_Auth_Validated, Access_Granted.
UAT-18	UC 7	Lottery Access Denied	Attempt to reserve tickets for a lottery event without being a verified winner.	ERROR Logs: Lottery_Auth_Failed, Access_Denied.
UAT-19	UC 7	Existing Order Check	Attempt to reserve tickets for an event while the visitor already holds an active order for it.	ERROR Logs: Reservation_Rejected_Existing_Order.

UAT-20	UC 7	Threshold Exceeded - Queue Created	Request to reserve tickets for an event where the load exceeds the threshold and no queue exists.	SUCCESS Logs: Load_Threshold_Exceeded, Queue_Initialized, User_Queued.
UAT-21	UC 7	Queue Turn Arrives	Wait in the virtual queue until the system grants turn authorization	SUCCESS Logs: Queue_Turn_Granted, Auth_Issued
UAT-22	UC 7	Tickets Unavailable	Select tickets that were concurrently locked by another user just moments before.	ERROR Logs: Reservation_Failed_Unavailable.
UAT-23	UC 8	Manage Order - Remove Ticket	Submit request to remove ticket from order a specific ticket from the active order before the timer expires.	SUCCESS Logs: Order_Updated, Ticket_Released.
UAT-24	UC 8	Active Order Timeout	Wait without performing any action until the active order countdown timer reaches zero.	FAIL Logs: Order_Timeout, Tickets_Released.
UAT-25	UC 8	Manage Expired Order	Attempt to remove a ticket or change quantity after the active order countdown timer has already reached zero	ERROR Logs: Order_Modification_Failed, Order_Expired.

UAT-26	UC 9	Successful Checkout	Submit valid payment details and request to finalize purchase within the active order time limit.	SUCCESS Logs: Payment_Success, Tickets_Issued.
UAT-27	UC 9	Checkout Policy Violation	Attempt to checkout with a ticket quantity exceeding the event's purchase policy limit.	ERROR Logs: Checkout_Failed_Policy_Violation.
UAT-28	UC 9	Payment Declined	Submit payment details that are rejected by the external payment gateway.	FAIL Logs: Payment_Declined.
UAT-29	UC 9	Payment Gateway Timeout	Submit valid payment details but simulate a network timeout or lack of response from the external payment gateway.	ERROR Logs: Payment_Timeout, Checkout_Aborted.
UAT-30	UC 9	Supply System Failure	Successfully process payment, but simulate a crash or disconnect in the ticket supply system.	FAIL Logs: Supply_Failed, Refund_Issued, Tickets_Released.
UAT-31	UC 10	Resume Active Order	Log in as Member, add tickets to active order, log out. Log back in before the timer expires.	SUCCESS Logs:Logout_Active_Order_Saved, Login_Order_Resumeed.

UAT-32	UC 10	Active Order Timeout	Log in, add tickets to active order, log out. Wait for the timer to expire, then log in.	FAIL Logs: Order_Timer_Expired, Tickets_Released.
UAT-33	UC 11	Open Company Success	Log in as Member, submit valid company registration details.	SUCCESS Logs: Company_Created, Role_Founder_Assigned.
UAT-34	UC 11	Open Company Duplicate	Submit company registration with an already existing company name.	ERROR Logs: Company_Creation_Failed, Error_Duplicate_Name.
UAT-35	UC 12	View History (Data)	Log in as a Member with past purchases and navigate to history.	SUCCESS Logs: History_Retrieved_Success.
UAT-36	UC 12	View History (Empty)	Log in as a Member with no purchases and navigate to history.	SUCCESS Logs: History_Retrieved_Empty.
UAT-37	UC 13	Update Profile Success	Submit valid updated identifying details in the profile page.	SUCCESS Logs: Profile_Updated_Success.

UAT-38	UC 13	Update Profile Fail	Submit details with invalid formatting (e.g., malformed email).	ERROR Logs: Profile_Update_Failed, Error_Invalid_Format.
UAT-39	UC 14	Lottery Registration	Submit registration request for lottery on an eligible event while logged in.	SUCCESS Logs: User_Added_To_Lottery_Pool.
UAT-40	UC 14	Lottery Win Dispatch	Trigger a lottery win via the system for the registered Member.	SUCCESS Logs: Lottery_Win_Dispatched, Auth_Code_Generated.
UAT-41	UC 15	Successful Event Creation	Log in as Owner, submit valid event details with venue map, zones, and pricing.	SUCCESS Logs: Event_Created, Venue_Map_Configured.
UAT-42	UC 15	Missing Required Fields	Submit event creation form with missing mandatory fields (e.g., no date or no venue map).	ERROR Logs: Event_Creation_Failed, Error_Missing_Fields.
UAT-43	UC 15	Inactive Company	Attempt to create an event under a suspended production	ERROR Logs:

			company.	Event_Creation_Blocked, Error_Inactive_Company.
UAT-44	UC 16	Set Purchase Policy	Define a purchase policy limiting max 4 tickets per buyer for an event.	SUCCESS Logs: Purchase_Policy_Set
UAT-45	UC 16	Set Visible Discount	Create an Early Bird visible discount (e.g., 20% off until a specific date).	SUCCESS Logs: Discount_Policy_Set , Type_Visible.
UAT-46	UC 16	Set Coupon Code	Create a hidden coupon code with a fixed discount amount.	SUCCESS Logs: Coupon_Created.
UAT-47	UC 16	Invalid Coupon at Checkout	A buyer enters a non-existent coupon code during checkout.	ERROR Logs: Coupon_Validation_Failed.
UAT-48	UC 16	Policy Conflict	Set a conditional discount that contradicts the purchase policy (e.g., “buy 5 get 1 free” but max tickets is 4).	ERROR Logs: Policy_Conflict_Detected.
UAT-49	UC 16	Modify Policy With Active Orders	Change the purchase policy for an event while visitors currently have active orders	WARNING Logs: Policy_Update_Warning_Active_

			with locked tickets.	Orders, Policy_Updated.
UAT-50	UC 17	Appoint Manager	Founder appoints a Member as Manager with selected permissions.	SUCCESS Logs: Manager_Appointed, Permissions_Set.
UAT-51	UC 17	Appoint Owner	The founder appoints a Member as the owner.	SUCCESS Logs: Owner_Appointed.
UAT-52	UC 17	Non-Founder Appoints Owner	An Owner (not Founder) attempts to appoint another Owner.	ERROR Logs: Appointment_Denied, Error_Not_Founder.
UAT-53	UC 17	Remove Owner	Founder removes an existing Owner from the company.	SUCCESS Logs: Owner_Removed, Permissions_Revoked.
UAT-54	UC 17	Owner Resigns	An Owner voluntarily resigns from their role in the company.	SUCCESS Logs: Owner_Resigned, Founder_Notified.
UAT-55	UC 18	Report With Data	The owner generates a sales report for an event with past purchases.	SUCCESS Logs: Report_Generated.

UAT-56	UC 18	Report Empty	The owner generates a report for an event with zero sales.	SUCCESS Logs: Report_Generated_Empty.
UAT-57	UC 19	Suspend Success	Founder suspends a company with no future events.	SUCCESS Logs: Company_Suspended, Notifications_Dispatched.
UAT-58	UC 19	Suspend With Sold Tickets	Founder suspends a company that has future events with sold tickets.	WARNING Logs: Suspension_Warning_Active_Events.
UAT-59	UC 19	Reopen Success	Founder reopens a company they previously suspended.	SUCCESS Logs: Company_Reopened, Notifications_Dispatched.
UAT-60	UC 19	Reopen Admin-Closed	Founder attempts to reopen a company that was closed by a System Admin.	ERROR Logs: Reopen_Denied, Error_Admin_Closure.
UAT-61	UC 20	Authorized Action Success	Log in as Manager with “edit events” permission. Edit an existing event’s details.	SUCCESS Logs: Action_Authorized, Event_Updated.

UAT-62	UC 20	Unauthorized Action Denied	Log in as Manager without “edit policies” permission. Attempt to modify the purchase policy.	ERROR Logs: Action_Denied, Error_Missing_Permission.
UAT-63	UC 20	Permission Revoked Mid-Action	Manager begins editing an event. The owner revokes the Manager’s “edit events” permission before the Manager submits.	ERROR Logs: Permission_Revoked , Action_Denied_Mid_Session.
UAT-64	UC 21	Admin Close Company	Log in as Admin and execute closure for an active company.	SUCCESS Logs: Company_Closed, Roles_Revoked, Notifications_Dispatched.
UAT-65	UC 21	Admin Close Fail	Execute closure command for a non-existent company_id.	ERROR Logs: Closure_Failed, Error_Company_Not_Found.
UAT-66	UC 22	Admin Ban Member	Admin executes ban command on an active Member account.	SUCCESS Logs: Account_Disabled, Roles_Revoked_Globally.
UAT-67	UC 22	Ban Sole Founder	Ban a Member who is the sole founder of an active company.	WARNING Logs:

				Account_Disabled, Warning_Orphaned_ Company.
UAT-68	UC 23	Global History Query	Admin applies a filter (e.g., specific event) with existing records.	SUCCESS Logs: Global_History_ Queried_Success.
UAT-69	UC 23	Global History Empty	Admin applies a filter with no matching records.	SUCCESS Logs: Global_History_ Queried_Empty.